Sincronizarea firelor de execuție

Andrei Toma | Membru 1 | Grupa CR-242

Echipa: Andrei Toma și Ganenco Bogdan. Un singur proiect Java, cu patru fire de lucru, creat și pornit din clasa Main.

Scop și sarcină

Studierea sincronizării prin metodele clasei Thread. Varianta 6: Th1 și Th2 calculează suma produselor valorilor de pe poziții impare, două câte două, în sens direct și invers. Th3 parcurge intervalul [654, 1278], iar Th4 parcurge [123, 908] de la sfârșit.

Se păstrează convenția proiectului de la laboratorul 1: pozițiile sunt indicii Java, numărați de la 0. Indicii impari sunt 1, 3, ..., 99. Tabloul are 100 de valori între 1 și 100, generate cu seed-ul 6 pentru rezultate repetabile.

Contribuția individuală

Fișier propriu: Student1Threads.java. Th1 formează perechile (1,3), (5,7), ..., (97,99). Th2 le parcurge invers: (99,97), (95,93), ..., (3,1). Produsul fiecărei perechi se adaugă unei sume locale de tip long.

Metode: join() și sleep(). Th1 așteaptă terminarea lui Th2 prin join(). Funcția comună scrieLent() folosește sleep(100), astfel încât literele numelui și prenumelui apar la interval de 100 ms.

Ordine și resurse comune

Calculele se execută concurent. Fiecare fir construiește un jurnal local, apoi îl afișează în ordinea Th2 -> Th1 -> Th4 -> Th3. Textele sunt: Toma, Andrei, CR-242, Programarea concurenta si distribuita. Astfel, literele nu se amestecă între fire.

Main creează exact patru fire de lucru și le așteaptă cu join(). Swing are propriul fir grafic. Nu se folosesc synchronized, wait/notify sau clase de sincronizare din java.util.concurrent pentru cele patru fire.

Cod pregătit cu asistență AI. Explicațiile și contribuția individuală trebuie înțelese pentru susținere.

Cod comun: Main.java

```
import java.awt.GraphicsEnvironment;
import java.util.Arrays;
import java.util.Random;
import java.util.function.Consumer;

public class Main {
    public static void main(String[] args) throws Exception {
        boolean consola = Arrays.asList(args).contains("--console");
        LabWindow fereastra = consola || GraphicsEnvironment.isHeadless()
                ? null : LabWindow.deschide();
        Consumer<String> output = text -> {
            System.out.print(text);
            System.out.flush();
            if (fereastra != null) fereastra.adauga(text);
        };

        int[] mas = new int[100];
        Random random = new Random(6);
        for (int i = 0; i < mas.length; i++) mas[i] = random.nextInt(100) + 1;
        output.accept("LAB 3 | VARIANTA 6 | CR-242\nAndrei Toma + Ganenco Bogdan\n");
        output.accept("mas[0..49]  = " + Arrays.toString(Arrays.copyOfRange(mas, 0, 50)) + "\n");
        output.accept("mas[50..99] = " + Arrays.toString(Arrays.copyOfRange(mas, 50, 100)) + "\n");
        output.accept("Pozitii impare = indici Java 1, 3, ..., 99.\n\n");

        // Main creeaza toate cele patru fire; fiecare membru are clasa sa.
        Thread[] fire = new Thread[4];
        fire[0] = new Thread(new Student1Threads(mas, false, fire, output), "Th1");
        fire[1] = new Thread(new Student1Threads(mas, true, fire, output), "Th2");
        fire[2] = new Thread(new Student2Threads(false, fire, output), "Th3");
        fire[3] = new Thread(new Student2Threads(true, fire, output), "Th4");

        // Pornim intai firele asteptate, pentru ca isAlive() sa fie relevant.
        for (int i : new int[] {1, 0, 3, 2}) fire[i].start();
        for (Thread fir : fire) fir.join();
        output.accept("\nFINAL: toate cele 4 fire s-au terminat.\n");
    }

    public static void scrieLent(String text, Consumer<String> output)
            throws InterruptedException {
        for (int i = 0; i < text.length(); i++) {
            output.accept(String.valueOf(text.charAt(i)));
            Thread.sleep(100);
        }
        output.accept("\n");
    }
}
```

Cod propriu: membrul 1

```
import java.util.function.Consumer;

/** Membru 1: Andrei Toma. Th1 si Th2, sincronizare prin join() si sleep(). */
public class Student1Threads implements Runnable {
    private final int[] mas;
    private final boolean invers;
    private final Thread[] fire;
    private final Consumer<String> output;

    public Student1Threads(int[] mas, boolean invers, Thread[] fire,
                           Consumer<String> output) {
        this.mas = mas;
        this.invers = invers;
        this.fire = fire;
        this.output = output;
    }

    @Override
    public void run() {
        String nume = Thread.currentThread().getName();
        StringBuilder jurnal = new StringBuilder("\n--- " + nume + " | produse pe perechi ---\n");
        long suma = 0;
        int perechi = 0;
        int pas = invers ? -4 : 4;
        // Aceleasi 25 de perechi: (1,3), (5,7), ... sau (99,97), (95,93), ...
        for (int i = invers ? 99 : 1; invers ? i >= 3 : i <= 97; i += pas) {
            int j = i + (invers ? -2 : 2);
            long produs = (long) mas[i] * mas[j];
            suma += produs;
            perechi++;
            jurnal.append(String.format(
                    "%s: mas[%d]=%d * mas[%d]=%d => produs=%d; suma=%d%n",
                    nume, i, mas[i], j, mas[j], produs, suma));
        }
        jurnal.append(nume + ": perechi=" + perechi + "; suma finala=" + suma + "\n");
        try {
            // Th1 asteapta terminarea lui Th2, inclusiv afisarea numelui de familie.
            if (!invers) fire[1].join();
            output.accept(jurnal.toString());
            output.accept(nume + (invers ? " | Numele: " : " | Prenumele: "));
            Main.scrieLent(invers ? "Toma" : "Andrei", output);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Rezultate și verificare

Rezultate obținute la rularea proiectului complet

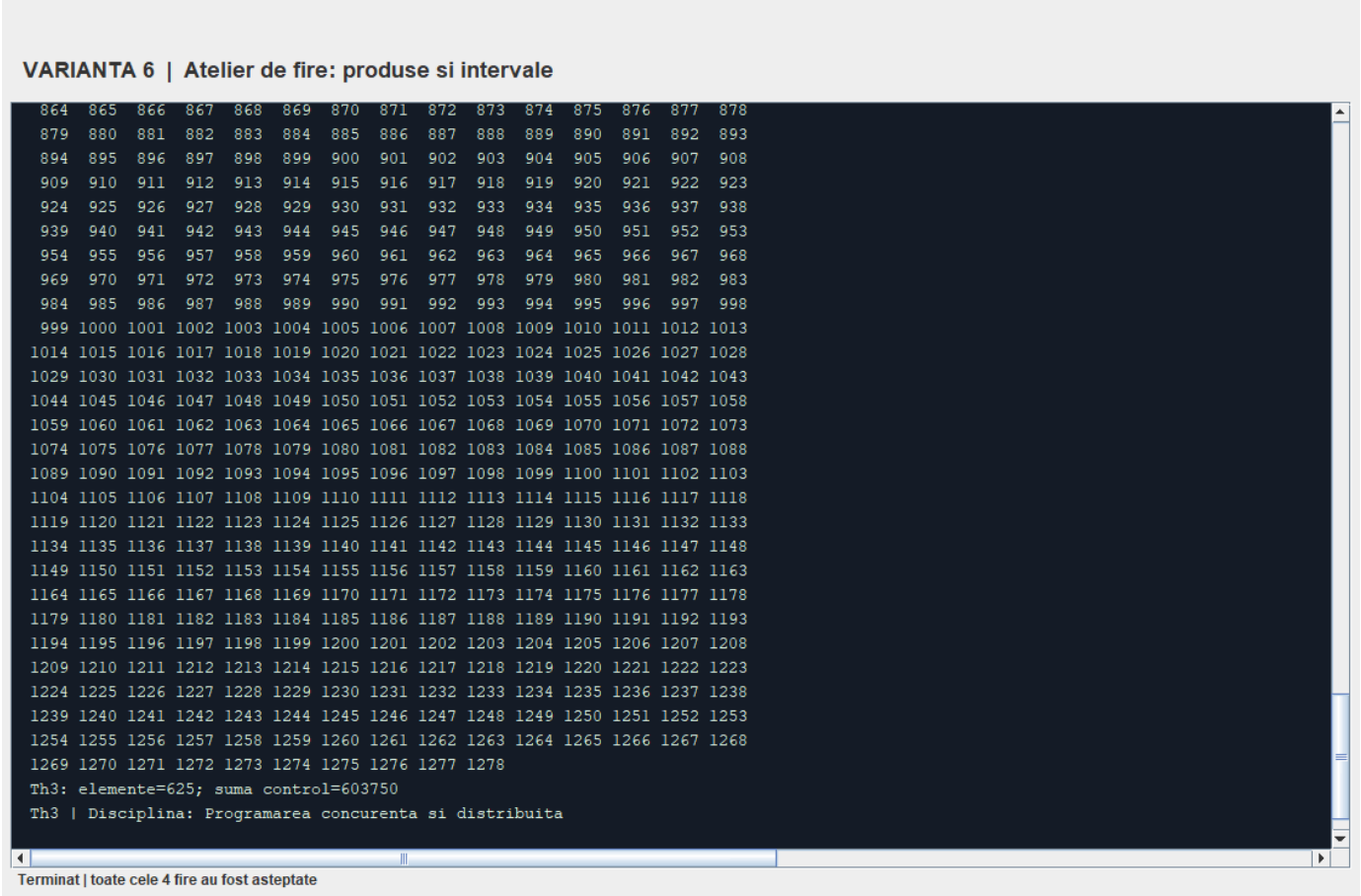
Th1 și Th2: câte 25 de perechi; aceeași sumă finală: 65926.

Th3: 625 de valori, prima 654, ultima 1278; suma 603750.

Th4: 786 de valori, prima 908, ultima 123; suma 405183.

Verificare: toate produsele și sumele cumulative, fiecare valoare din intervale și ordinea textelor au fost comparate cu un calcul independent. Compilarea și rularea s-au încheiat fără erori.

Interfața aplicației



Imagine randată direct din fereastra aplicației după rularea completă; jurnalul se derulează pentru a vedea toate sarcinile.

Concluzie și pornire

Sincronizarea stabilește ordinea afișării fără a modifica sarcinile. Datele de calcul sunt locale sau doar citite. interrupt() trezește un fir, iar join() și isAlive() urmăresc terminarea efectivă.

```
javac -encoding UTF-8 -d Lab3/out Lab3/src/*.java
java -cp Lab3/out Main
```